

Rezolvare Completă și Detaliată

Examenul Național pentru Definitivare în Învățământ

Anul 2024 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Definitivat Model 2024	2
SUBIECTUL I (60 de puncte)	2
1. Algoritmul lui Dijkstra (Grafuri Ponderate)	2
a) Noțiuni preliminare:	2
b) Descrierea algoritmului Dijkstra pe un graf cu 7 noduri	2
c) Problemă completă (Dijkstra)	3
2. Tehnologia Informației (TIC): Formatarea la nivel de pagină	4
3. Programare: Cifră de control	4
4. Baze de date: Organizație de ciclism	6
II. Rezolvare Definitivat Varianta 2 (24 iulie 2024)	7
SUBIECTUL I (60 de puncte)	7
1. Tablouri bidimensionale (Matrice)	7
a) Exemple de declarare	7
b) Accesul la elemente și relațiile pe diagonale	8
c) Problemă completă (Suma elementelor de sub diagonala principală)	8
2. Dispozitive de stocare amovibile/detașabile	8
3. Programare: k-perechi	9
4. Baze de date: Emisiuni Radio	11
SUBIECTUL al II-lea (30 de puncte)	12
1. Proiectarea unei activități didactice: Asaltul de idei (Brainstorming)	12
2. Test practic (Greedy)	12

I. Rezolvare Definitivat Model 2024

SUBIECTUL I (60 de puncte)

1. Algoritmul lui Dijkstra (Grafuri Ponderate)

a) Noțiuni preliminare:

- **Graf ponderat:** Un graf în care fiecărei muchii/arc i se asociază un număr real numit pondere sau cost.
- **Drum:** O succesiune de noduri în care fiecare două noduri consecutive sunt unite printr-o muchie/arc.
- **Cost al unui drum:** Suma ponderilor muchiilor/arcelor componente.

b) Descrierea algoritmului Dijkstra pe un graf cu 7 noduri

Fie graful orientat cu nodurile $\{1, 2, 3, 4, 5, 6, 7\}$ și nodul de start 1. Ponderile arcelor sunt: $(1, 2) : 2, (1, 3) : 4, (2, 3) : 1, (2, 4) : 7, (3, 5) : 3, (4, 6) : 1, (5, 4) : 2, (5, 7) : 5, (6, 7) : 2$.

Etapele algoritmului Dijkstra:

1. Inițializăm vectorul de distanțe d cu ∞ pentru toate nodurile, cu excepția nodului de start 1, pentru care $d[1] = 0$. Inițializăm o mulțime de noduri selectate $S = \emptyset$.
2. La fiecare pas i :
 - Alegem un nod u neselectat cu $d[u]$ minim.
 - Îl adăugăm în S .
 - Pentru fiecare vecin v al lui u , relaxăm arcul (u, v) : $d[v] = \min(d[v], d[u] + \text{cost}(u, v))$.
3. Repetăm pasul 2 de $N - 1$ ori.

Trace-ul algoritmului pentru graful ales:

- **Inițial:** $d = (0, \infty, \infty, \infty, \infty, \infty, \infty)$
- **Pas 1:** Selectăm nodul 1. Vecini: 2, 3.
 - $d[2] = \min(\infty, 0 + 2) = 2$
 - $d[3] = \min(\infty, 0 + 4) = 4$
 - $d = (0, 2, 4, \infty, \infty, \infty, \infty)$
- **Pas 2:** Selectăm nodul 2. Vecini: 3, 4.
 - $d[3] = \min(4, 2 + 1) = 3$
 - $d[4] = \min(\infty, 2 + 7) = 9$
 - $d = (0, 2, 3, 9, \infty, \infty, \infty)$
- **Pas 3:** Selectăm nodul 3. Vecin: 5.
 - $d[5] = \min(\infty, 3 + 3) = 6$
 - $d = (0, 2, 3, 9, 6, \infty, \infty)$
- **Pas 4:** Selectăm nodul 5. Vecini: 4, 7.
 - $d[4] = \min(9, 6 + 2) = 8$
 - $d[7] = \min(\infty, 6 + 5) = 11$
 - $d = (0, 2, 3, 8, 6, \infty, 11)$
- **Pas 5:** Selectăm nodul 4. Vecin: 6.
 - $d[6] = \min(\infty, 8 + 1) = 9$
 - $d = (0, 2, 3, 8, 6, 9, 11)$
- **Pas 6:** Selectăm nodul 6. Vecin: 7.
 - $d[7] = \min(11, 9 + 2) = 11$ (rămâne neschimbat)
 - $d = (0, 2, 3, 8, 6, 9, 11)$

- **Pas 7:** Selectăm nodul 7. Distanțele finale sunt $d = (0, 2, 3, 8, 6, 9, 11)$.

c) Problemă completă (Dijkstra)

- **Enunț:** Să se determine drumul de cost minim de la nodul de start 1 la toate celelalte noduri într-un graf ponderat cu n noduri și m arce.

• **Cod C++:**

```
#include <iostream>
#include <vector>
using namespace std;
const int INF = 1000000000;
int w[100][100], d[100];
bool viz[100];
int main() {
    int n, m, u, v, c;
    cin >> n >> m;
    for (int i = 1; i <= n; ++i) {
        d[i] = INF; viz[i] = false;
        for (int j = 1; j <= n; ++j) w[i][j] = INF;
    }
    for (int i = 0; i < m; ++i) {
        cin >> u >> v >> c; w[u][v] = c;
    }
    d[1] = 0;
    for (int pas = 1; pas <= n; ++pas) {
        int min_val = INF, u = -1;
        for (int i = 1; i <= n; ++i)
            if (!viz[i] && d[i] < min_val) { min_val = d[i]; u = i; }
        if (u == -1) break;
        viz[u] = true;
        for (int v = 1; v <= n; ++v)
            if (w[u][v] != INF && d[u] + w[u][v] < d[v]) d[v] = d[u] + w[u][v];
    }
    for (int i = 1; i <= n; ++i) cout << d[i] << " ";
    return 0;
}
```

• **Cod Pascal:**

```
program Dijkstra;
const INF = 1000000000;
var
    n, m, u, v, c, i, j, pas, min_val, min_u: integer;
    w: array[1..100, 1..100] of integer;
    d: array[1..100] of integer;
    viz: array[1..100] of boolean;
begin
    read(n, m);
    for i := 1 to n do
        begin
            d[i] := INF; viz[i] := false;
            for j := 1 to n do w[i, j] := INF;
        end;
    for i := 1 to m do
        begin
            read(u, v, c); w[u, v] := c;
```

```

end;
d[1] := 0;
for pas := 1 to n do
begin
    min_val := INF; min_u := -1;
    for i := 1 to n do
        if (not viz[i]) and (d[i] < min_val) then
            begin
                min_val := d[i]; min_u := i;
            end;
    if min_u = -1 then break;
    viz[min_u] := true;
    for v := 1 to n do
        if (w[min_u, v] <> INF) and (d[min_u] + w[min_u, v] < d[v]) then
            d[v] := d[min_u] + w[min_u, v];
    end;
    for i := 1 to n do write(d[i], ' ');
end.

```

2. Tehnologia Informației (TIC): Formatarea la nivel de pagină

- **Noțiuni preliminare:** Un document text este compus din secțiuni, pagini, paragrafe și caractere. Elemente de interfață într-un procesor de text: Rigla (ruler), bara de stare, meniul de configurare.
 - **Personalizare pagină:**
 1. **Margini:** Ajustarea spațiului gol de pe margini (Top, Bottom, Left, Right, Gutter).
 2. **Orientare:** Portret (vertical) sau Peisaj (orizontal).
 3. **Dimensiune hârtie:** Ex. A4, Letter, A3.
 4. **Coloane:** Divizarea textului paginii pe mai multe coloane.
 5. **Antet și subsol (Header & Footer):** Zona destinată titlurilor sau numerotării.
 6. **Fundal pagină:** Culori de fundal sau filigrane (watermarks).
-

3. Programare: Cifra de control

- **Descriere:** Subprogramul suma(n) calculează suma cifrelor unui număr natural. Algoritmul din programul principal procesează elementele din fișierul def2023.in, le determină cifra de control în mod repetat și contorizează minimul obținut.

Soluție C++:

```

#include <iostream>
#include <fstream>
using namespace std;

int suma(long long n) {
    int s = 0;
    while (n > 0) {
        s += n % 10;
        n /= 10;
    }
    return s;
}

```

```
int cifraControl(long long n) {
    if (n == 0) return 0;
    long long s = n;
    while (s >= 10) {
        s = suma(s);
    }
    return s;
}

int main() {
    ifstream fin("def2023.in");
    long long x;
    int min_cifra = 10;
    int count_min = 0;
    while (fin >> x) {
        int cc = cifraControl(x);
        if (cc < min_cifra) {
            min_cifra = cc;
            count_min = 1;
        } else if (cc == min_cifra) {
            count_min++;
        }
    }
    fin.close();
    cout << min_cifra << " " << count_min << endl;
    return 0;
}
```

Soluție Pascal:

```
program CifraDeControl;
var
    fin: text;
    x: int64;
    min_cifra, count_min, cc: integer;

function suma(n: int64): integer;
var s: integer;
begin
    s := 0;
    while n > 0 do
        begin
            s := s + (n mod 10);
            n := n div 10;
        end;
    suma := s;
end;

function cifraControl(n: int64): integer;
var s: int64;
begin
    if n = 0 then cifraControl := 0
    else
        begin
            s := n;
            while s >= 10 do
                s := suma(s);
            end;
        end;
    cifraControl := s;
end;
```

```

        cifraControl := s;
    end;
end;

begin
    assign(fin, 'def2023.in');
    reset(fin);
    min_cifra := 10;
    count_min := 0;
    while not eof(fin) do
        begin
            read(fin, x);
            cc := cifraControl(x);
            if cc < min_cifra then
                begin
                    min_cifra := cc;
                    count_min := 1;
                end
            else if cc = min_cifra then
                count_min := count_min + 1;
            end;
            close(fin);
            writeln(min_cifra, ' ', count_min);
        end.
    end.

```

4. Baze de date: Organizație de ciclism

Model conceptual:

- COMPETITIE: id_competitie, denumire, oras_secretariat, telefon_contact, email_contact, site_web.
- TIP_PROBA: id_tip_proba, denumire_tip, descriere_generala.
- PROBA: id_proba, id_competitie, id_tip_proba, descriere_particulara, data_desfasurare, categorie_participanti.

Relații și reguli:

- COMPETITIE 1:M PROBA; o competiție poate avea mai multe probe.
- TIP_PROBA 1:M PROBA; un tip de probă poate apărea în competiții diferite.
- **1NF**: toate attributele sunt atomice.
- **2NF**: attributele non-cheie depind complet de cheia primară a tabelului.
- **3NF**: descrierea generală a tipului de probă este în TIP_PROBA, nu repetată în PROBA.
- O competiție poate avea mai multe probe de același tip doar dacă diferă categoria participanților sau data/descrierea particulară, conform regulilor organizației.

Model fizic:

```

CREATE TABLE COMPETITIE (
    id_competitie INT PRIMARY KEY AUTO_INCREMENT,
    denumire VARCHAR(150) NOT NULL,
    oras_secretariat VARCHAR(100) NOT NULL,
    telefon_contact VARCHAR(30),
    email_contact VARCHAR(120),
    site_web VARCHAR(150)
);

```

```
CREATE TABLE TIP_PROBA (
    id_tip_proba INT PRIMARY KEY AUTO_INCREMENT,
    denumire_tip VARCHAR(60) NOT NULL UNIQUE,
    descriere_generala TEXT
);

CREATE TABLE PROBA (
    id_proba INT PRIMARY KEY AUTO_INCREMENT,
    id_competitie INT NOT NULL,
    id_tip_proba INT NOT NULL,
    descriere_particulara TEXT,
    data_desfasurare DATE NOT NULL,
    categorie_participanti VARCHAR(60) NOT NULL,
    FOREIGN KEY (id_competitie) REFERENCES COMPETITIE(id_competitie),
    FOREIGN KEY (id_tip_proba) REFERENCES TIP_PROBA(id_tip_proba)
);
```

SQL pentru competiții cu cel puțin o probă în anul curent:

```
SELECT DISTINCT c.denumire
FROM COMPETITIE c
JOIN PROBA p ON c.id_competitie = p.id_competitie
WHERE YEAR(p.data_desfasurare) = YEAR(CURDATE());
```

II. Rezolvare Definitivat Varianta 2 (24 iulie 2024)

SUBIECTUL I (60 de puncte)

1. Tablouri bidimensionale (Matrice)

a) Exemple de declarare

1. Declarare cu inițializare cu date (alocare statică):

• C++:

```
int A[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

• Pascal:

```
const
    A: array[1..3, 1..3] of integer = (
        (1, 2, 3),
        (4, 5, 6),
        (7, 8, 9)
    );
```

2. Declarare cu alocare dinamică de memorie:

• C++:

```
int** A = new int*[linii];
for(int i = 0; i < linii; ++i) {
    A[i] = new int[coloane];
}
```

• **Pascal:**

```
type
  TLinie = array[1..100] of integer;
  PMatrice = ^array[1..100] of ^TLinie;
```

b) Accesul la elemente și relațiile pe diagonale

Fie o matrice pătratică A cu indicii de linie i și coloană j (1-indexed de la 1 la n):

- **Accesul:** A[i][j] (C++) sau A[i, j] (Pascal).
- **Diagonala Principală:** $i = j$.
 - Deasupra (dreapta): $i < j$.
 - Dedesubt (stânga): $i > j$.
- **Diagonala Secundară:** $i + j = n + 1$.
 - Deasupra (stânga): $i + j < n + 1$.
 - Dedesubt (dreapta): $i + j > n + 1$.

c) Problemă completă (Suma elementelor de sub diagonala principală)

- **Enunț:** Să se calculeze suma elementelor situate strict dedesubtul diagonalei principale a unei matrice pătratice de dimensiune n.

• **Cod C++:**

```
#include <iostream>
using namespace std;
int main() {
    int n, A[100][100], sum = 0;
    cin >> n;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j) {
            cin >> A[i][j];
            if (i > j) sum += A[i][j];
        }
    cout << "Suma: " << sum << endl;
    return 0;
}
```

• **Cod Pascal:**

```
program SumaMatrice;
var
  n, i, j, sum: integer;
  A: array[1..100, 1..100] of integer;
begin
  read(n);
  sum := 0;
  for i := 1 to n do
    for j := 1 to n do
      begin
        read(A[i, j]);
        if i > j then sum := sum + A[i, j];
      end;
    writeln('Suma: ', sum);
  end.
```

2. Dispozitive de stocare amovibile/detașabile

- **Memorie internă vs externă:** Memoria internă (RAM, Cache) este rapidă, volatilă și direct accesibilă procesorului. Memoria externă (HDD, SSD, Stick USB) este nevolatilă,

are capacitate mare și păstrează datele pe termen lung, comunicând cu procesorul prin intermediul controlerelor și magistralelor de date.

- **Parametri:**

1. **Viteza de transfer (Read/Write Speed):** Exprimată în MB/s sau GB/s. Determină rapiditatea accesului la date.
2. **Capacitatea:** Volumul maxim de date stocat (GB/TB).

- **Dispozitive amovibile:**

1. **Stick USB (Flash Drive):** Stocare pe cipuri de memorie Flash (EEPROM). Conectare prin port USB. Avantaj: Portabilitate extremă.
2. **SSD Extern:** Stocare pe cipuri NAND Flash. Conectare prin USB-C sau Thunderbolt. Avantaj: Viteze mari de transfer.

3. Programare: k-perechi

- **Descriere:** Subprogramul `nrDiv` calculează numărul de divizori ai lui n în $O(\sqrt{n})$. Programul principal citește intervalul, ordonează capetele crescător și parcurge elementele consecutive, contorizând perechile pentru care ambele numere au exact k divizori.

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <algorithm>
using namespace std;

int nrDiv(int n) {
    int cnt = 0;
    for (int d = 1; d * d <= n; ++d) {
        if (n % d == 0) {
            cnt++;
            if (d * d < n) cnt++;
        }
    }
    return cnt;
}

int main() {
    ifstream fin("def2024.in");
    int k, x, y;
    if (!(fin >> k >> x >> y)) return 0;
    fin.close();

    int st = min(x, y);
    int dr = max(x, y);

    int count_pairs = 0;
    if (st < dr) {
        int prev_divs = nrDiv(st);
        for (int i = st; i < dr; ++i) {
            int curr_divs = nrDiv(i + 1);
            if (prev_divs == k && curr_divs == k) {
                count_pairs++;
            }
            prev_divs = curr_divs;
        }
    }
}
```

```

    }

    cout << count_pairs << endl;
    return 0;
}

```

Soluție Pascal:

```

program KPerechi;
uses math;

var
    fin: text;
    k, x, y, st, dr, i, prev_divs, curr_divs, count_pairs: integer;

function nrDiv(n: integer): integer;
var
    d, cnt: integer;
begin
    cnt := 0;
    d := 1;
    while d * d <= n do
    begin
        if n mod d = 0 then
        begin
            cnt := cnt + 1;
            if d * d < n then
                cnt := cnt + 1;
            end;
            d := d + 1;
        end;
        nrDiv := cnt;
    end;

begin
    assign(fin, 'def2024.in');
    reset(fin);
    read(fin, k, x, y);
    close(fin);

    st := min(x, y);
    dr := max(x, y);
    count_pairs := 0;

    if st < dr then
    begin
        prev_divs := nrDiv(st);
        for i := st to dr - 1 do
        begin
            curr_divs := nrDiv(i + 1);
            if (prev_divs = k) and (curr_divs = k) then
                count_pairs := count_pairs + 1;
            prev_divs := curr_divs;
        end;
    end;
end;

```

```
writeln(count_pairs);
end.
```

4. Baze de date: Emisiuni Radio

Model conceptual:

- PIESA: id_piesa, titlu, compozitor, interpret, gen_muzical, an_aparitie, durata_secunde.
- EMISIUNE: id_emisiune, denumire, frecventa, data_start_grila, data_sfarsit_grila, ora_difuzare, minut_difuzare, descriere.
- DIFUZARE: id_difuzare, data_difuzare, ora_difuzare, minut_difuzare, leagă o piesă de o emisiune.

Relații:

- O piesă poate fi difuzată de mai multe ori, în cadrul mai multor emisiuni: PIESA 1:M DIFUZARE.
- O emisiune poate difuza mai multe piese în timp: EMISIUNE 1:M DIFUZARE.
- Relația M:N dintre piese și emisiuni este normalizată prin tabela asociativă DIFUZARE.

Restricții și forme normale:

- 1FN:** attribute atomice; ora și minutul sunt câmpuri separate sau o valoare temporală unică, nu liste de ore.
- 2FN:** toate attributele non-cheie depind de cheia primară a tabelului în care se află.
- 3FN:** datele piesei nu se repetă în DIFUZARE, iar datele emisiunii nu se repetă în PIESA.
- durata_secunde > 0, an_aparitie <= YEAR(CURDATE()), id_piesa și id_emisiune din DIFUZARE sunt chei străine valide.

Model fizic:

Tabel	Câmp	Tip	Rol / restricții
PIESA	id_piesa	INT	PK, AUTO_INCREMENT
	titlu	VARCHAR(150)	NOT NULL
	compozitor	VARCHAR(120)	NOT NULL
	interpret	VARCHAR(150)	NOT NULL
	gen_muzical	VARCHAR(60)	
	an_aparitie	INT	CHECK
	durata_secunde	INT	CHECK > 0
EMISIUNE	id_emisiune	INT	PK, AUTO_INCREMENT
	denumire	VARCHAR(120)	NOT NULL
	frecventa	VARCHAR(30)	săptămânală/cotidiană/etc.
	data_start_grila	DATE	
	data_sfarsit_grila	DATE	NULL permis
	ora_difuzare	INT	0..23
	minut_difuzare	INT	0..59
DIFUZARE	descriere	TEXT	NULL
	id_difuzare	INT	PK, AUTO_INCREMENT
	id_piesa	INT	FK -> PIESA
	id_emisiune	INT	FK -> EMISIUNE

	data_difuzare	DATE	NOT NULL
	ora_difuzare	INT	0..23
	minut_difuzare	INT	0..59

SQL pentru afișarea cronologică cerută:

```
SELECT d.data_difuzare, d.ora_difuzare, d.minut_difuzare
FROM DIFUZARE d
JOIN PIESA p ON d.id_piesa = p.id_piesa
WHERE p.titlu = 'Anotimpurile - Vara'
      AND p.compozitor = 'Antonio Vivaldi'
      AND YEAR(d.data_ora) = YEAR(CURDATE())
ORDER BY d.data_difuzare ASC, d.ora_difuzare ASC, d.minut_difuzare ASC;
```

SUBIECTUL al II-lea (30 de puncte)

1. Proiectarea unei activități didactice: Asaltul de idei (Brainstorming)

- **Secvența A (Greedy).**
- **Caracteristici ale brainstormingului:**
 1. Separă etapa de generare a ideilor de etapa de evaluare, astfel încât elevii propun liber strategii de alegere.
 2. Stimulează participarea activă și valorifică răspunsurile multiple, inclusiv soluțiile inițial incomplete.
 3. Este util pentru compararea metodelor, deoarece ideile elevilor pot fi ulterior testate după corectitudine și eficiență.
- **Formă de organizare:** activitate pe grupe mici, urmată de discuție frontală.
- **Mijloc de învățământ:** tablă/proiector și fișă de lucru cu o problemă de optimizare.
- **Activitate de învățare:** identificarea unei strategii Greedy pentru selecția activităților compatibile.
- **Scenariu:**
 - Profesorul prezintă problema: se dau activități cu ore de început/sfârșit, se cere numărul maxim de activități compatibile.
 - Elevii propun criterii: activitatea cea mai scurtă, cea care începe prima, cea care se termină prima, cea cu durată maximă.
 - Profesorul notează toate ideile fără validare imediată, apoi cere exemple/contraexemple.
 - Elevii testează criteriile pe același set de date și observă că alegerea după timpul minim de finalizare conduce la soluție optimă.
 - Profesorul formalizează regula Greedy și discută complexitatea după sortarea activităților.

2. Test practic (Greedy)

Test practic - 90 puncte + 10 puncte din oficiu

1. **Item 1 (30p):** Se citesc n activități, fiecare cu ora de început și ora de sfârșit. Implementați un algoritm Greedy care afișează numărul maxim de activități compatibile.
 - Răspuns așteptat: sortare după ora de sfârșit, selectarea primei activități compatibile cu ultima aleasă.
 - Barem: citire date 4p; sortare corectă 8p; selecție Greedy corectă 12p; afișare 3p; corectitudine globală 3p.

2. **Item 2 (30p):** Pentru problema rucsacului fracționar, calculați valoarea maximă care poate fi transportată pentru o capacitate dată, folosind obiecte cu greutate și profit.
- Răspuns așteptat: calcul raport profit/greutate, sortare descrescătoare, preluare integrală sau fracționară.
 - Barem: calcul raport 6p; sortare 8p; tratarea obiectelor întregi 8p; tratarea fracției finale 5p; afișare 3p.
3. **Item 3 (30p):** Explicați de ce strategia „aleg activitatea care se termină cel mai devreme” este eficientă pentru selecția activităților și precizați complexitatea.
- Răspuns așteptat: alegerea locală lasă interval maxim pentru activitățile următoare; după sortare, parcurgerea este liniară.
 - Barem: justificare corectă a alegerii Greedy 12p; explicație a pașilor algoritmului 8p; complexitate $O(n \log n)$ datorată sortării 6p; claritate 4p.